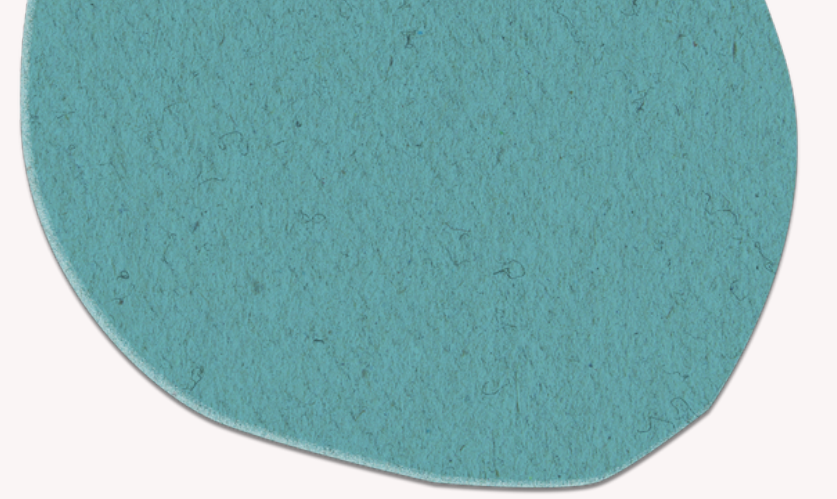
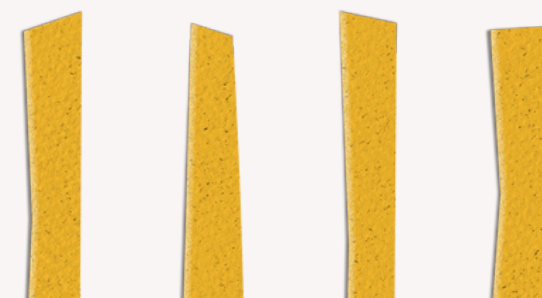
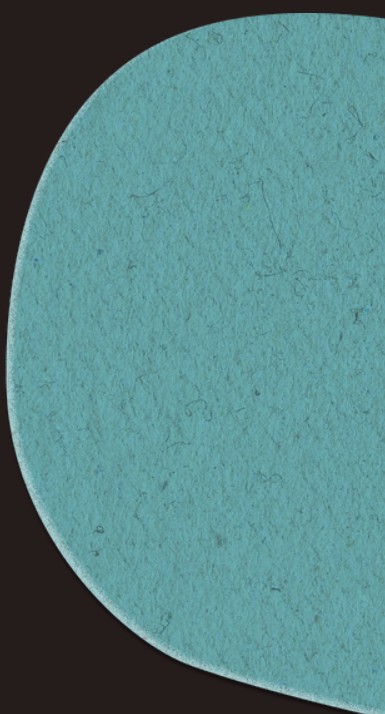




EDU-TACK

Python @ Universidad_Central





ALCANCE

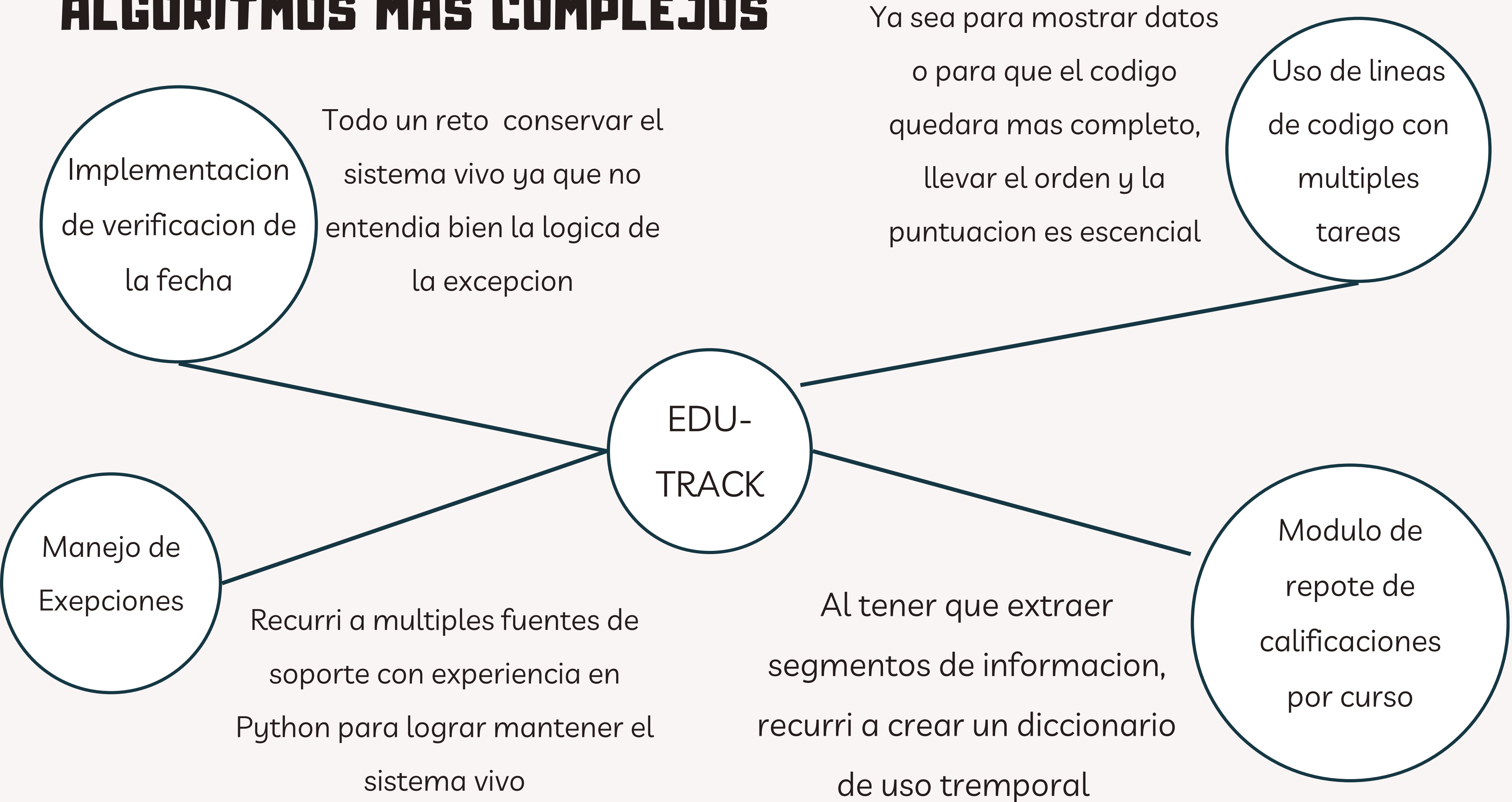
Logros

- Acceso controlado
- Manejo de estudiantes
 - Agregar, ver todos y buscar
- Manejo de cursos
 - Agregar, ver todos y buscar
- Manejo de calificaciones
 - Agregar y ver todas las calificaciones
- Generacion de reportes
 - De estudiantes y promedio por curso

Limitaciones

- Se mantiene solo en version codigo
 - El manejo de exepciones se puede mejorar
 - Se puede implementar un super usuario para agregar usuarios y dejar registro de quien hizo la gestion.
- 

ALGORITMOS MAS COMPLEJOS




```

# gestion de calificaciones
def agregar_calificacion(self, cedula, codigo_curso, calificacion, fecha): # metodo para agregar calificaciones
    estudiante = self.buscar_estudiante(cedula) # definicion de variables para uso interno
    curso = self.buscar_curso(codigo=codigo_curso)

    if estudiante and curso:
        # try:
        #     calificacion = float(calificacion)
        #     if not (0 <= calificacion <= 100):
        #         raise ValueError
        #     fecha = datetime.strptime(fecha, "%Y-%m-%d").date()
        # except ValueError:
        #     print('Formato de calificacion o fecha invalido. Use el formato tal como se solicita.')
        #     return

        calificacion = { # creacion de diccionario nuevo con las llaves
            "cedula": cedula,
            "codigo_curso": codigo_curso,
            "calificacion": calificacion,
            "fecha": fecha # confiamos en el usuario para ingresar un dato correcto
        }
        self.calificaciones.append(calificacion) # se agrega el diccionario con la calificacion
        print('\n>>> Calificacion registrada con exito! <<<')
    else:
        print('\n Parece que ingresaste un dato incorrecto, intente de nuevo')

# gestion de reportes
def reporte_calificaciones_por_curso(self): # metodo de
    print("\n--- Reporte de Calificaciones por Curso ---")
    if not self.cursos:
        print("No hay cursos registrados.")
        return

    if not self.calificaciones:
        print("\n>>> No hay calificaciones registradas para los cursos. <<<") # revision de existencia de calificaciones registradas
        return

    curso_dict = {} # se crea un diccionario de uso interno y se itera sobre calificaciones
    for calificacion in self.calificaciones:
        codigo_curso = calificacion['codigo_curso'] # obtiene codigo curso de calificacion
        if codigo_curso not in curso_dict: # sin o esta en el diccionario interno, lo agrega
            curso = self.buscar_curso(codigo=codigo_curso)
            curso_dict[codigo_curso] = { # agrega el curso y los estudiantes al diccionario nuevo
                'nombre': curso['nombre'] if curso else 'Desconocido', # si desconocido es porque tiene basura
                'estudiantes': []
            }
        estudiante = self.buscar_estudiante(calificacion['cedula']) # busca los datos de cada estudiante
        curso_dict[codigo_curso]['estudiantes'].append({ # agrega varios datos del estudiante al diccionario nuevo
            'nombre': f"{estudiante['nombre']} {estudiante['apellidos']}" if estudiante else 'Desconocido',
            'calificacion': calificacion['calificacion'],
            'fecha': calificacion['fecha']
        })

    for codigo, info in curso_dict.items(): # se obtiene el nombre del curso del diccionario nuevo
        print(f"\nCurso: {info['nombre']}")
        for estudiante_info in info['estudiantes']: # se obtiene el reatande de informacion de la lista estudiantes
            print(f"Estudiante: {estudiante_info['nombre']}")
            print(f"Calificacion: {estudiante_info['calificacion']}")
            print(f"Fecha: {estudiante_info['fecha']}")
        print("\n Reporte generado con exto! <<<")

```

```

def lista_cursos(self): # metodo para mostrar todos los cursos
    print(' >>> No hay cursos agregados en el sistema <<< ') # revision de existencia de cursos
    else:
        for curso in self.cursos: # muestra los datos d elos cursos registrados
            print(f"\nCodigo: {curso['codigo']}")
            print(f"Nombre: {curso['nombre']}")
            print(f"Descripcion: {curso['descripcion']}")
            print(f"Profesor: {curso['profesor']}")
            print(f'\n >>> Fin de la informacion del curso. <<< ')

def buscar_curso(self, **kwargs): # metodo para buscar entre todos los cursos
    codigo = kwargs.get('codigo') # por codigo o por nombre
    nombre = kwargs.get('nombre')

    for curso in self.cursos:
        if (codigo and curso['codigo'] == codigo) or (nombre and curso['nombre'].lower() == nombre.lower()): #.lower mantiene simplicidad a nivel de sistema
            print('\n-- Curso encontrado --')
            print(f"\nCodigo: {curso['codigo']}") # muestra los datos del curso buscado
            print(f"Nombre: {curso['nombre']}")
            print(f"Descripcion: {curso['descripcion']}")
            print(f"Profesor: {curso['profesor']}")
            return curso

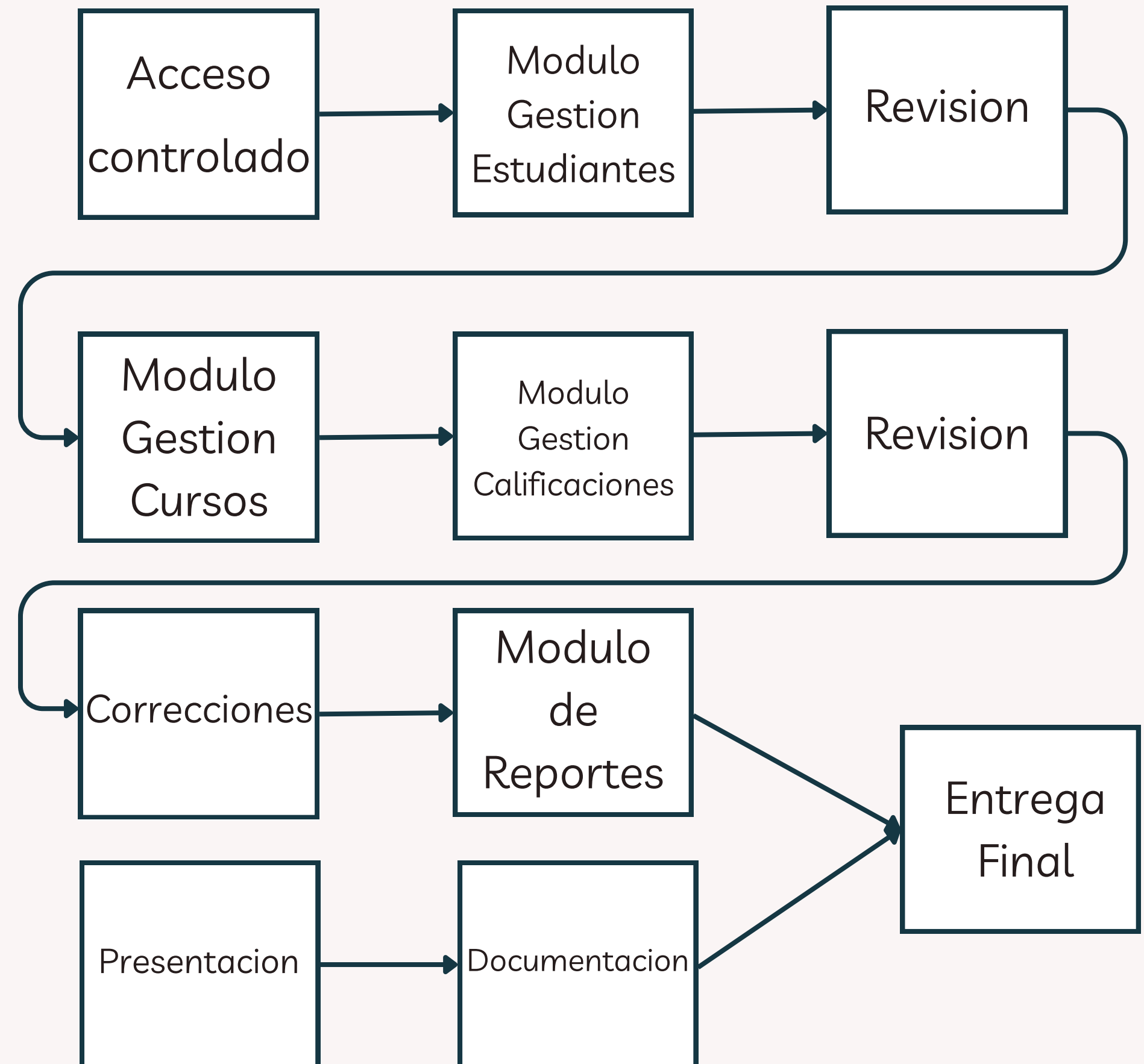
    print("\n>>> Curso no encontrado, intentelo de nuevo. <<<") # revision de existencia del curso
    return None

```

DESCRIPCION DEL CASO

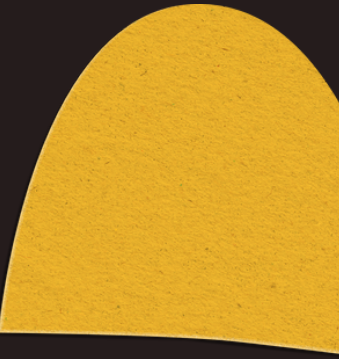
Se solocita desarrollar un sistema que permita la gestion, control y administracion de estudiantes, cursos y calificaciones para la institucion educativa "EDUTRACK". El sistema debe permitir a los empleados de la institucion administrar estudiantes, cursos y registrar calificaciones de manera interactiva.

El programa se desarrolla en etapas





PROPUESTA



- Enfoque en el cliente, manteniendo la funcionalidad en cada segmento del programa.
- Se manejan situaciones de ingresos inesperados por parte del usuario.
- Se mantiene una estructura y explicación fácil de entender en cada instrucción compleja.
- Se evalúa el formato del código, archivos y nombres respectivos del programa y contenidos.





GRACIAS!

